

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



STUDENT SCIENTIFIC RESEARCH REPORT

Faculty of Information Technology

SKETCH-BASED 3D ANIMATION GENERATION

Supervisor: TS. Ma Th Cháu

Student Group:

Lỗ Minh Tóm 22024500 K67I-IS

Hanoi, 2025

Tóm tắt nội dung

Sketch-based modeling systems simplify the creation of 3D characters, but animating them remains a challenging task due to the complexity of rigging and secondary motion control. Traditional animation workflows require extensive manual effort to define skeletal structures and dynamic motion, making the process time-consuming and technically demanding. This creates a gap between creative expression and technical challenges. To address the problem, we propose an interactive framework that combines real-time skeletonization with secondary motion generation using a mass-spring system. Our system includes three key functions: (i) the user sketches a character, (ii) the system automatically generates a rigged skeleton, and (iii) the system applies dynamic animations without the need for manual keyframing. The system combines the best aspects of two methods: the simplicity and accessibility of sketch-based modeling, and the expressive, physics-driven secondary motion provided by the mass-spring system. Beginner animators can create expressive, physics-driven animations effortlessly without tedious adjustments.

Keywords: Sketch-based Modeling, Skeletonization, Mass-spring System, Real-time Animation.

Mc lc

1	Gii thiu	2
2	Background and Notation	3
2.1	Related Works	3
2.2	Real-time Skeletonization for Sketch-based Modeling	4
2.2.1	Local Sub-Skeleton Extraction	5
2.2.2	Sub-Skeleton Connection	5
2.2.3	Global Skeleton Refinement	6
2.3	Fast Simulation of Mass-spring Systems	7
2.3.1	Fundamental Theory	7
2.3.2	Reformulating the Energy Potential	9
3	Method	12
3.1	System Overview	12
3.2	Functions	14
3.3	Implementation	15
3.3.1	Building Matrices	15
3.3.2	Local Update	16
3.3.3	Global Update	17
3.3.4	Convergence Check	18
4	Results and Discussion	19
4.1	Experimental Setup	19
4.2	Dynamic Motion Effects	19
4.3	Effects of Stiffness Parameter	21
4.4	Complexity	21
5	Conclusion	22

Chng 1

Gii thiu

Hu ht còc hnh thc biu t ngh thut u tri dji t tòngh quy chun n tòngh t do. Vò d: th ca bao gm t nhng th th cú lut l kht khe ùi hi s trau chut t m, cho n li c ng bin t do c sòng tòngh ngay trong lữc trnh din. Tng t, óm nhc cú th lị mt bn son tho cht ch hoc mt bui hũa nhc ngu hng; din viỏn cú th hc thuc li thoi hoc din xut ng bin trc tip; vị mt ha s cú th định nhũu thòng cho mt bc sn du, hoc ch mt 30 giỏy cho mt nỏt phòc ha c ng nhanh.

im chung ca còc hnh thc t do lị tòngh nhanh chóng, tc thớ, mang tòngh bn nng vị ờt ri ro, cho phỏp ngh s to ra mt kt qu hoịn chnh ngay t con s khũng. Còch thc biu t nịy úng vai trũ then cht trong quò trnh sòng to: con ngi t mc sòng to cao nhũt khi trong trng thòi thng hoa mt trng thòi ùi hi s gii phúng khi còc rịo cn k thut vị c bit lị khũng b gũ bú bi ni s mc sai lm.

Tuy nhĩn, loi hnh ngh thut hot hnh thc s cha bao gi cú c s t do ú. Ngay c nhng k thut n gin nhũt cng cc k tn thi gian, c bit lị trong khũng gian ba chiu, bi cú quò nhũu bc trung gian t dng hnh, thit lp khung xng vị còc thũng s bin dng, cho n to dòng vị t khung hnh then cht ng gia y tng vị kt qu cui cũng. Chc chn cha cú cũng c nịo cho phỏp mt ngi lịm hot hnh i t mt trang giy trng n mt nhón vt ba chiu chũyn ng hoịn chnh trong thi gian thc. Vic thiu ht mt ch sòng to ngu hng c thứ ò buc còc ngh s hot hnh phi thc hìn quò trnh tòm tũi ca mnh thũng qua còc phng tin khòc nh v phòc tho tay, xỏy dng kch bn hnh nh hoc quay video thc t.

Chng 2

Background and Notation

2.1 Related Works

Sketch-based Modeling The pioneering sketch-based system, Teddy [1], significantly simplifies shape modeling by allowing users to create 3D forms with just a few strokes. Over time, numerous follow-up works [2–6] have expanded on this approach, enabling the creation of more complex shapes and making sketch-based modeling more comprehensive. These systems offer a fun and intuitive experience for novice users, making 3D modeling more accessible. However, their simplicity can be a drawback for professionals who require precise control over model accuracy.

Rigging and Skeletonization In modern character animation pipelines, a rig consists of a skeleton, a cycle-free graph where nodes represent joints and edges represent bones. Skin weights define how the model’s surface deforms in response to skeletal movement [7]. Because of the complexity of the manual process, automatic skeletonization and rigging techniques have been extensively researched. Given a static character mesh and a generic skeleton, Pinocchio [8] adapts the skeleton to the character and attaches it to the surface, allowing skeletal motion data to animate the character. RigMesh [9] extends this concept by creating a rig for each sketched part in real-time, and update the rig as parts are merged or cut. More recent approaches have leveraged deep learning for automation. RigNet [10] introduces a neural network that predicts a skeleton that matches the animator expectations in joint placement and topology. For single-image rigging, Object Wake-up [11] reconstructs 3D objects and generates skeletons from 2D images using deep implicit functions. Specifically for sketch-based modeling, RealSkel [12] provides a real-time skeletonization technique. Despite these advancements, automatic skeletonization and rigging alone is not sufficient for high-quality animation. Proper rigging still requires refinement, such as adjusting joint positions, fine-tuning skin weights, and incorporating inverse

kinematics (IK).

Mass-spring System Mass-spring systems have long been used in both scientific computing and physics-based animation to simulate movement and deformation [13]. These systems represent objects as networks of interconnected masses and springs, enabling realistic simulations of elasticity, bending, and other physical behaviors. Early models for cloth simulation [14, 15] demonstrated how simple physics-based structures could produce visually convincing motion. They have also been applied to approximate the deformation of volumetric solids [16] and linear elements such as hair [17, 18]. Spring bone dynamics, as seen in tools like Cartoon Animator 5 [19] and VRM [20], provide simplified secondary motion effects, making them valuable for game development and AR/VR applications. Researchers have also improved these methods for greater stability and efficiency, with the Fast Mass-Spring method [21] introducing a more efficient solver for real-time deformable object simulation.

After reviewing related works, we find that RealSkel [12] and the Fast Mass-Spring method [21] align well with our research objectives. These methods fulfill key requirements, including real-time skeletonization to quickly generate an animatable skeleton from sketches, and efficient mass-spring simulations for responsive motion. Therefore, in the following sections, we examine the fundamental concepts behind these two influential works. A clear understanding of their methodologies is essential for implementing our system and effectively integrating their strengths.

2.2 Real-time Skeletonization for Sketch-based Modeling

It is necessary to examine how RealSkel [12] constructs a real-time skeleton. We begin with some key definitions before diving into the methodology.

Definition 1 (Simple Polygon). A simple polygon is a polygon that does not intersect itself and has no holes.

Definition 2 (Straight Skeleton) A straight skeleton is a special partitioning of a polygon into monotone regions traced by a continuous inward offsetting of the contour edge. [22]

The method consists of three main steps: local sub-skeleton extraction, sub-skeleton connection, and global skeleton refinement. First, the local skeleton is extracted from the processed polygon stroke, forming a subpart along with the corresponding sub-mesh. Next, individual sub-skeletons are connected based on their intersections and the sequential order in which the subparts were created. Finally, a global refinement

process allows users to adjust the skeleton structure with coarse-to-fine control, ensuring an optimized and animation-ready hierarchy.

2.2.1 Local Sub-Skeleton Extraction

Efficient real-time skeletonization in sketch-based modeling requires an immediate and structured approach to extracting a local sub-skeleton. Traditional methods often rely on full mesh construction before skeleton extraction, leading to computational inefficiencies. The proposed method circumvents this issue by employing a three-step process: polygon acquisition, straight skeleton extraction, and skeleton simplification.

The process begins with the conversion of raw sketch strokes into a simple polygon. The sketch line undergoes uniform discretization to smooth noise, followed by Douglas-Peucker (DP) simplification, which approximates the shape with fewer points while maintaining structural integrity. The resultant polygon is then oriented counter-clockwise to facilitate the next phase.

The straight skeleton extraction follows, leveraging an inward offset propagation technique that traces skeleton structures by computing bisectors of polygon edges. Two primary events occur during propagation: Edge Events, where edges collide and vanish, and Split Events, where edges intersect and subdivide, forming a refined medial structure. This approach ensures computational efficiency compared to traditional mesh-based or voxel-based skeletonization.

Finally, the skeleton simplification step removes redundant vertices and edges, ensuring an optimized animatable structure. To address over-simplification and preserve geometric fidelity, the authors introduce BoundedDP, an enhanced DP variant. BoundedDP integrates both axis curvature and shape variation metrics. The resulting skeleton is not only concise but also directly suitable for animation, demonstrating the efficiency of this local extraction method in real-time applications.

2.2.2 Sub-Skeleton Connection

A critical challenge in real-time skeletonization for sketch-based modeling lies in establishing hierarchical connections between independently generated sub-skeletons. The proposed method integrates a real-time sub-skeleton connection process that dynamically links new skeletal parts to existing structures based on geometric intersection tests.

The connection process relies on a cylinder-ball approximation, where each skeleton segment is represented as a general cylinder, and joints are treated as inscribed balls. This representation enables fast intersection testing between newly created and existing subparts, allowing real-time detection of attachment points. When two

subparts intersect, their corresponding skeletons are hierarchically linked based on Euclidean distance between child joints and parent bones.

To ensure anatomical accuracy, the algorithm distinguishes between two attachment types: Bone Split Events and Joint Connect Events. If a child joint falls within a parent bones influence region, the bone is split, and a new joint is introduced for connection. Otherwise, if the child joint is closer to an existing parent joint, it is directly connected without modifying the parent bone structure. This ensures that the hierarchy maintains both structural coherence and geometric adaptability.

The connection process also enforces a tree-structured hierarchy, where the initially created subpart serves as the root, and subsequent additions form a parent-child relationship. This approach ensures automatic organization of skeleton complexity, allowing real-time modifications to the hierarchy based on user interactions. The integration of this sub-skeleton connection method significantly improves efficiency and usability in sketch-based modeling environments.

2.2.3 Global Skeleton Refinement

To achieve an animation-ready skeleton, the raw sub-skeleton structure must undergo a global refinement process that removes redundancies and optimizes bone hierarchy. The proposed method implements a multi-level control strategy that provides users with flexible operations at three hierarchical levels: branch level, subpart level, and global level.

At the branch level, skeleton complexity is adjusted using BoundedDP-based curve simplification, which allows for localized control over individual skeletal branches. Users can dynamically tune the number of joints along a skeletal axis, ensuring efficient representation of both coarse and fine-grained structures.

At the subpart level, the system introduces three refinement operations:

1. **Joints Merging:** Clustered junction nodes are consolidated into single nodes, reducing redundancy.
2. **Branch Pruning:** Short, insignificant branches are removed to enhance skeletal clarity.
3. **Edge Collapsing:** Short bones between internal joints are merged to streamline the skeleton structure.

At the global level, users can adjust resolution parameters to control overall skeletal complexity. Threshold parameters (e.g., ϵ_s , ϵ_m) enable coarse-to-fine adjustments, allowing users to prioritize detail in critical regions (e.g., hands) while simplifying others (e.g., torso). This flexibility ensures the skeleton adapts to diverse animation

needs, balancing computational efficiency with artistic control. The refinement process is fully automated but remains user-guided, ensuring the final output is both structurally optimal and animation-ready.

2.3 Fast Simulation of Mass-spring Systems

This section reviews the key contributions by [21], which introduces a fast and stable method for time integration of mass-spring systems using a block coordinate descent solver. Their approach reformulates the implicit Euler method as an energy minimization problem, introducing spring directions as auxiliary unknowns to decouple nonlinear terms and allow efficient pre-factorization of the system matrix.

2.3.1 Fundamental Theory

Before delving into the details of their proposed method, we first clarify some fundamental theory underlying mass-spring systems and implicit time integration.

Consider the model as a graph consisting of m points in 3D space:

- Each vertex p_i represents a point mass.
- Each edge represents a spring.

In a mass-spring simulation, each point starts with an initial position and initial velocity, which determine how it moves over time. According to Newtonian physics, if the current state of an object is known, its future state can be calculated by applying physical equations. This principle also holds in computer simulations, where the system updates positions and velocities at discrete time steps to approximate real-world motion. The key question to be addressed here is: **How can we determine the next positions of the vertices after a time step h ?**

The system evolves through a discrete sequence of time steps $t_1, t_2, \dots$ with a constant time step h . At each time step t_n , the system's configuration is represented as $\mathbf{p}_n \in \mathbb{R}^{3m}$, $\mathbf{p}_n$ is a vector containing the positions of all m points in 3D space at time t_n .

Assume that at time t , the current state $\mathbf{p}_t$ is known. For $t = 0$, these serve as the initial conditions for the entire simulation. For $t \geq 0$, they can still be considered the initial conditions for the subsequent time intervals. Denote $\mathbf{f}(\mathbf{p}_t)$ as the vector of forces acting on all particles $\mathbf{p}_i^t$ at time t , and $\mathbf{M} \in \mathbb{R}^{3m \times 3m}$ as the diagonal mass matrix.

Newton's Second Law states that the force $\mathbf{f} \in \mathbb{R}^3$ acting on an object is equal to

the product of its mass and acceleration $\mathbf{a} \in \mathbb{R}^3$

$$\mathbf{M}\mathbf{a} = \mathbf{f} \quad (2.1)$$

We construct the animation by applying this equation to every vertex in the graph. The force $\mathbf{f}_i$ acting on vertex i is the sum of all forces from adjacent springs ij .

In the real world, an object's trajectory follows a continuous curve over time. However, in this simulation, we are only concerned with the discrete positions of the vertices at specific time steps. We will construct discrete approximations using implicit Euler integration, including system states $\mathbf{p}$ (the position of each node) and velocity. The update rules are as follows:

$$\mathbf{p}_{n+1} = \mathbf{p}_n + h\mathbf{v}_{n+1} \quad (2.2)$$

$$\mathbf{v}_{n+1} = \mathbf{v}_n + h\mathbf{M}^{-1}\mathbf{f}(\mathbf{p}_{n+1}) \quad (2.3)$$

Using the position update equation from implicit Euler integration, we can express velocity in terms of discrete differences.

Backward Difference Approximation The velocity at time step n is given by:

$$h\mathbf{v}_n = \mathbf{p}_n - \mathbf{p}_{n-1} \quad (2.4)$$

Forward Difference Approximation Similarly, the velocity at the next time step $n + 1$ can be written as:

$$h\mathbf{v}_{n+1} = \mathbf{p}_{n+1} - \mathbf{p}_n \quad (2.5)$$

To eliminate velocity from the implicit Euler update, we multiply equation (2.3) by h and substitute equations (2.4) and (2.5), leading to:

$$\mathbf{p}_{n+1} - 2\mathbf{p}_n + \mathbf{p}_{n-1} = h^2\mathbf{M}^{-1}\mathbf{f}(\mathbf{p}_{n+1}) \quad (2.6)$$

Since $\mathbf{f}(\mathbf{p}_{n+1})$ depends on the unknown future position $\mathbf{p}_{n+1}$, this is called a nonlinear problem. The nonlinear system in equation (2.6) can be rewritten as an optimization problem. To simplify notation, define $\mathbf{x} := \mathbf{p}_{n+1}$ and $\mathbf{y} := 2\mathbf{p}_n - \mathbf{p}_{n-1}$. Multiplying both sides of equation (2.6) by the mass matrix $\mathbf{M}$, we obtain a more compact form:

$$\mathbf{M}(\mathbf{x} - \mathbf{y}) = h^2\mathbf{f}(\mathbf{x}) \quad (2.7)$$

2.3.2 Reformulating the Energy Potential

As stated in the paper by [21], the main idea of their technique is to reformulate the energy potential E in a way that will allow us to employ a block coordinate descent method. In a mass-spring system, the elastic potential energy represents the energy stored in a deformed spring. According to Hooke's Law, the potential energy function is given by:

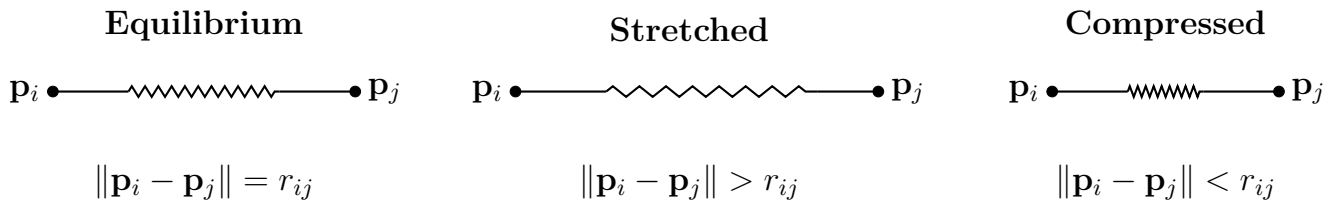
$$E(\mathbf{p}_i, \mathbf{p}_j) = \frac{1}{2}k(\|\mathbf{p}_i - \mathbf{p}_j\| - r_{ij})^2 \quad (2.8)$$

where:

- $E(\mathbf{p}_i, \mathbf{p}_j)$ is the elastic potential energy stored in the spring.
- k is the spring stiffness constant, which determines how strongly the spring resists deformation.
- $\mathbf{p}_i, \mathbf{p}_j$ are the positions of two connected mass points.
- r_{ij} is the rest length of the spring, i.e., the natural length when the spring is neither stretched nor compressed.

With these definitions in mind, we can now analyze how the potential energy behaves under different conditions (see Figure 2.1).

- When $\|\mathbf{p}_i - \mathbf{p}_j\| = r_{ij}$, the energy is **zero**, meaning the spring is at equilibrium.
- When $\|\mathbf{p}_i - \mathbf{p}_j\| > r_{ij}$, the spring is **stretched**, storing positive potential energy.
- When $\|\mathbf{p}_i - \mathbf{p}_j\| < r_{ij}$, the spring is **compressed**, also storing potential energy.



Honh 2.1: Three states of a mass-spring system: equilibrium, stretched, and compressed.

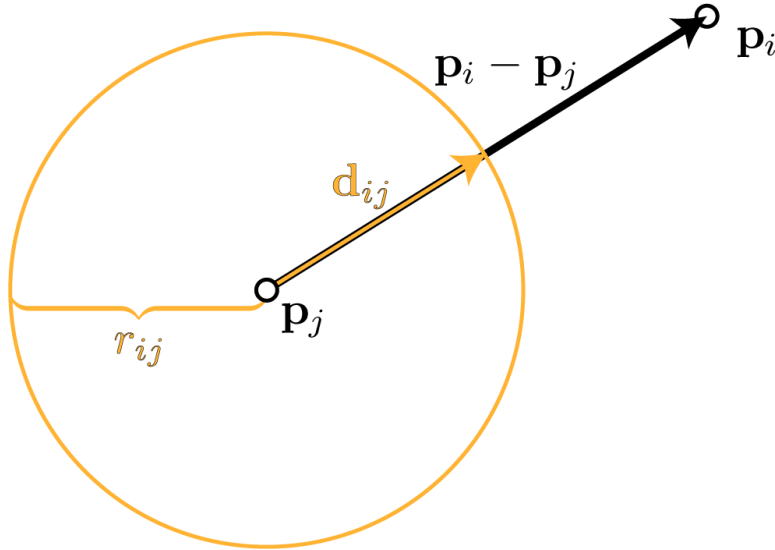
Because of the $\|\mathbf{p}_i - \mathbf{p}_j\|$ term, calculating E is a non-linear problem. The Euclidean distance between two points is given by:

$$\|\mathbf{p}_i - \mathbf{p}_j\| = \sqrt{(p_i^x - p_j^x)^2 + (p_i^y - p_j^y)^2 + (p_i^z - p_j^z)^2}$$

The presence of this square root function means that the energy function is not a simple quadratic function. The standard approach would be to apply gradient descent, Gauss-Newton method, or Newton's method. These methods, however, are either too slow or too complicated.

An interesting discovery has been made to simplify and accelerate the optimization algorithm. Specifically, for each spring, the nonlinear energy function can be reformulated as a smaller, independent optimization problem. This means, we need to find the vector of length that is as close as possible to the current spring vector (see Figure 2.2).

$$(\|\mathbf{p}_i - \mathbf{p}_j\| - r_{ij})^2 = \min_{\mathbf{d}_{ij} \in \mathbb{R}^3, \|\mathbf{d}\|=r_{ij}} \|(\mathbf{p}_i - \mathbf{p}_j) - \mathbf{d}_{ij}\|^2$$



Hình 2.2: Geometric interpretation of the optimization process. The vector $\mathbf{p}_i - \mathbf{p}_j$ represents the displacement between two connected points in the system. The rest length r_{ij} is shown as the radius of the reference circle. The updated vector $\mathbf{d}_{ij}$ (in orange) is computed to satisfy the spring constraint while maintaining the structural integrity of the system.

Having introduced the potential energy, we now revisit equation (2.7) for further analysis. Since $\mathbf{f}(\mathbf{x})$ represents the force, and force is the negative gradient of potential energy [23], we have:

$$\mathbf{f}(\mathbf{x}) = -\nabla E(\mathbf{x}) \quad (2.9)$$

Substituting this into equation (2.7) gives:

$$\mathbf{M}(\mathbf{x} - \mathbf{y}) = -h^2 \nabla E(\mathbf{x}) \quad (2.10)$$

This matches the critical point condition of the function:

$$g(\mathbf{x}) = \frac{1}{2}(\mathbf{x} - \mathbf{y})^T \mathbf{M}(\mathbf{x} - \mathbf{y}) + h^2 E(\mathbf{x}) \quad (2.11)$$

since setting $\nabla g(\mathbf{x}) = 0$ results in:

$$\mathbf{M}(\mathbf{x} - \mathbf{y}) + h^2 \nabla E(\mathbf{x}) = 0 \quad (2.12)$$

Thus, the solutions to equation (2.7) correspond to the critical points of the function given in (2.11). This leads to an optimization formulation $\min_{\mathbf{x}} g(\mathbf{x})$. This formulation, known as variational implicit Euler [24], allows solving the motion equations using optimization techniques instead of direct integration. This approach avoids numerical instability and large time-step restrictions that often arise in explicit integration methods [25].

From the method described in the paper [21], we know that the potential energy of the system can be written as:

$$E(\mathbf{x}) = \min_{\mathbf{d} \in U} \left(\frac{1}{2} \mathbf{x}^T \mathbf{L} \mathbf{x} - \mathbf{x}^T \mathbf{J} \mathbf{d} + \mathbf{x}^T \mathbf{f}_{\text{ext}} \right) \quad (2.13)$$

where $U = \{(\mathbf{d}_1, \dots, \mathbf{d}_s) \in \mathbb{R}^{2s} \mid \|\mathbf{d}_i\| = r_i\}$ represents the set of rest-length spring directions.

Now, substituting equation (2.13) into (2.11), we obtain:

$$\begin{aligned} g(\mathbf{x}) &= \frac{1}{2}(\mathbf{x} - \mathbf{y})^T \mathbf{M}(\mathbf{x} - \mathbf{y}) + h^2 \min_{\mathbf{d} \in U} \left(\frac{1}{2} \mathbf{x}^T \mathbf{L} \mathbf{x} - \mathbf{x}^T \mathbf{J} \mathbf{d} + \mathbf{x}^T \mathbf{f}_{\text{ext}} \right) \\ &= \min_{\mathbf{x} \in \mathbb{R}^{3m}, \mathbf{d} \in U} \left[\frac{1}{2} \mathbf{x}^T \mathbf{M} \mathbf{x} - \mathbf{x}^T \mathbf{M} \mathbf{y} + \frac{1}{2} \mathbf{y}^T \mathbf{M} \mathbf{y} + h^2 \left(\frac{1}{2} \mathbf{x}^T \mathbf{L} \mathbf{x} - \mathbf{x}^T \mathbf{J} \mathbf{d} + \mathbf{x}^T \mathbf{f}_{\text{ext}} \right) \right] \end{aligned} \quad (2.14)$$

Since we are minimizing with respect to $\mathbf{x}$, the constant term $\mathbf{y}^T \mathbf{M} \mathbf{y}$ can be ignored. Also, for simplification we introduce $\mathbf{b}$ as $\mathbf{b} = h^2 \mathbf{f}_{\text{ext}} - \mathbf{M} \mathbf{y}$. We obtain the following:

$$g(\mathbf{x}) = \min_{\mathbf{x} \in \mathbb{R}^{3m}, \mathbf{d} \in U} \left(\frac{1}{2} \mathbf{x}^T (\mathbf{M} + h^2 \mathbf{L}) \mathbf{x} - h^2 \mathbf{x}^T \mathbf{J} \mathbf{d} + \mathbf{x}^T \mathbf{b} \right) \quad (2.15)$$

Thus, equation (2.15) represents the final optimization problem, where the minimizer x^* provides the exact solution to the implicit Euler timestep.

Chng 3

Method

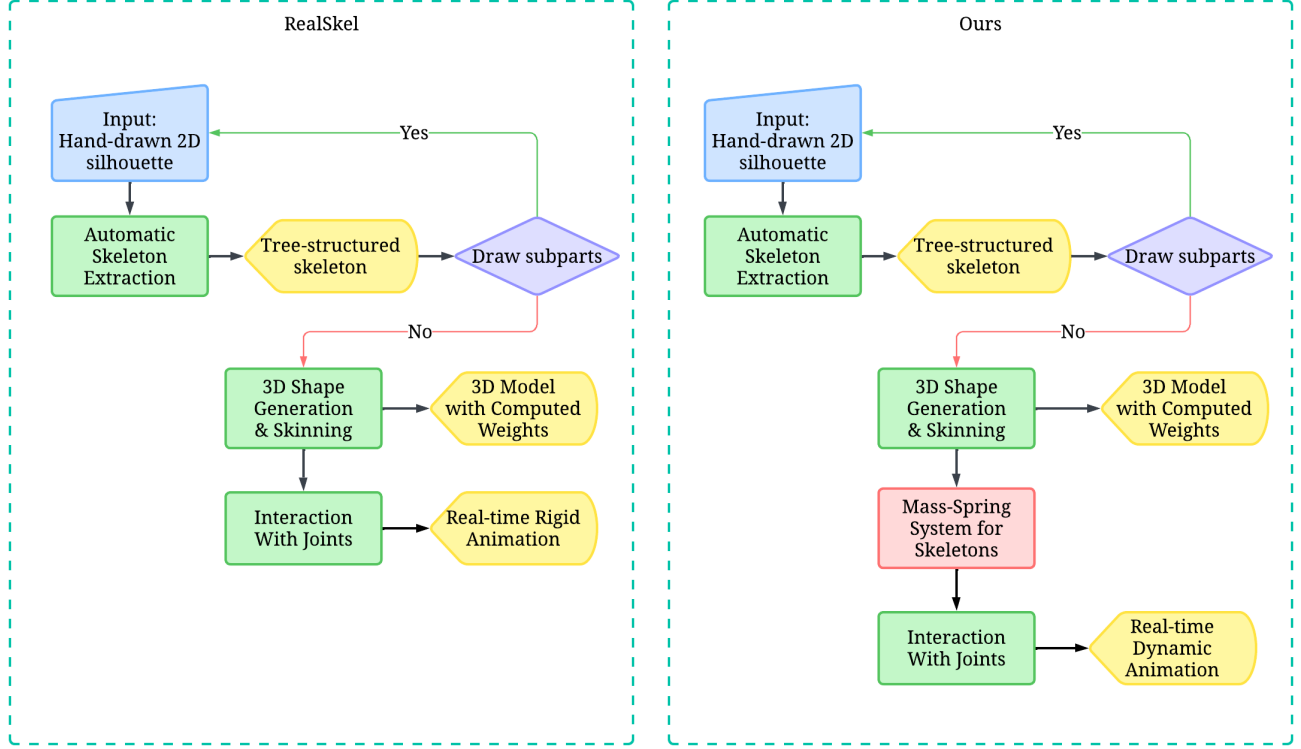
3.1 System Overview

Figure 3.1 highlights the key differences between the original RealSkel [12] system (left) and our proposed approach (right). Both frameworks follow a similar initial workflow: beginning with a hand-drawn 2D silhouette, performing automatic skeleton extraction, and optionally allowing users to add subparts.

In RealSkel, once the skeleton is generated and the shape undergoes skinning, users can directly interact with joints to produce real-time rigid animations. By contrast, our approach extends this workflow by integrating a Mass-Spring System for Skeletons after the skinning stage. This addition introduces secondary motion effects, allowing for real-time dynamic animation. In comparison, the original system applies strictly rigid transformations, lacking the dynamic movement achieved in our method.

We divide our system into three procedures:

1. **Automatic Skeleton Extraction:** The system first applies constrained Delaunay triangulation [26] to process the input stroke. Here, we define the user’s hand-drawn input as the silhouette of the model. The spine of the polygon, which also serves as the models skeleton, is then automatically determined using the method described by [22].
2. **3D Shape Generation & Skinning:** A 3D shape is then generated from the 2D silhouette using the approach outlined in [1]. Initially, at this stage, we elevate the vertices of the spine by an amount proportional to their distance from the polygon. The user, however, can adjust the model’s inflation by modifying the thickness parameter afterward. The system constructs a polygonal mesh that wraps around the spine and polygon, ensuring that sections form oval shapes and



Hình 3.1: Comparison of RealSkel and Our Approach. The original RealSkel system (left) directly applies rigid animation after skeleton extraction and skinning. Our approach (right) introduces a Mass-Spring System for Skeletons (highlighted in pink), adding secondary motion effects for real-time dynamic animation.

calculate. Skinning weights are then automatically computed using Bounded Bi-harmonic Weights (BBW) [27], preparing the model for skinning transformation via Linear Blend Skinning (LBS).

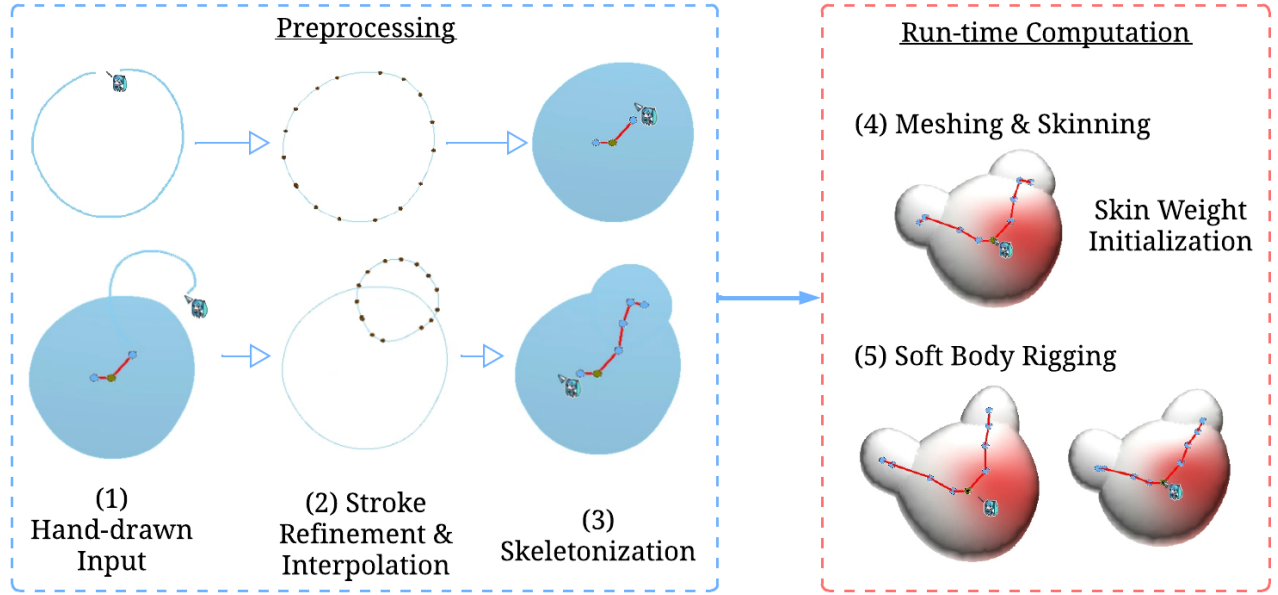
3. **Mass-spring System for Skeletons** To implement a mass-spring system for skeletal dynamics, we adapt the method proposed by [21], originally designed for cloth simulation, to a hierarchical skeletal structure. Unlike cloth, which is typically modeled as a network graph with interconnected nodes, a skeleton is inherently tree-structured, where each joint has a single parent and potentially multiple children. We treat each joint as a mass node and each bone as a spring to simulate skeletal dynamics. Although there are three types of springs: structural, bend and shear [28], in this study, we have not yet implemented shear and bend springs, which also play a crucial role in simulating more flexible deformations. Instead, we focus solely on structural springs to maintain bone length while introducing secondary motion effects.

The user can then draw additional subparts to expand the model, triggering a

new cycle of skeletonization, meshing, and skinning.

3.2 Functions

In this section, we present the system’s functionality from the user’s perspective (see Figure 3.2).



Hình 3.2: Overview of the sketch-based soft body rigging system. (1) The user provides a hand-drawn input to define the contour of the shape. (2) Stroke refinement and interpolation are performed to generate a cleaner representation. (3) A skeleton is extracted from the refined strokes to define the deformation structure. The first row illustrates the process when drawing a single part of the 3D model, while the second row shows the process of merging multiple sub-parts to create a larger model. (4) A meshing and skinning process assigns weights to the structure, generating a simulation mesh. (5) The system applies soft body rigging using a mass-spring system to enable physics-driven deformation. The final animation utilizes a combination of LBS for primary deformation and a Spring Bone system for secondary dynamic motion.

First, the user begins by providing a hand-drawn sketch, which serves as the initial input for the modeling process. The system then refines and interpolates the strokes to generate a smooth and consistent shape. Next, a skeleton is extracted from the refined strokes, defining the structural framework of the model. Once the skeleton is established, the user can press ‘D’ to initiate the meshing and skinning stage. The system automatically computes weight initialization based on the mesh

structure. Soft-body rigging is applied using a mass-spring system, enabling the model to exhibit dynamic deformations. The user can then select individual joints in the skeleton to manipulate, causing the entire structure to move dynamically as if connected by springs. Notably, when the user grabs a joint, it is considered fixed within the mass-spring system and is fully controlled by user movement for ease of use and convenience.

3.3 Implementation

Thanks to the work by [21], computing the mass-spring system state at each time step is simplified into a straightforward iterative process based on a local-global approach:

- **Step 1 (Local Update):** Given the current positions $\mathbf{p}$, determine d_{ij} for each spring.
- **Step 2 (Global Update):** Using all computed vectors d_{ij} , update $\mathbf{x}$ by minimizing the quadratic energy function described in equation (2.15).
- **Step 3 (Convergence Check):** If the solution has not yet converged, return to Step 1. If the maximum number of iterations is exceeded, terminate the algorithm.

We implemented our method using *C++* and the *Qt* framework [29]. *CMake* [30] is also required for build configuration and project management. The linear system was solved using the *Eigen* library [31].

The source code for the original RealSkel system is publicly available at: <https://github.com/jingma-git/RealSkel>.

Our implementation is available at: <https://github.com/maple1606/Sketch-to-Model>

3.3.1 Building Matrices

Before solving for the new positions of nodes, we construct three key matrices $\mathbf{L}$, $\mathbf{J}$ and $\mathbf{M}$ described in the original paper [21]. The process is detailed in Algorithm 1.

Algorithm 1 Construction of $\mathbf{L}$, $\mathbf{J}$, and $\mathbf{M}$

Require: Spring system with n points and s springs.

```
1: Initialize empty sparse matrices  $\mathbf{L}$ ,  $\mathbf{J}$ , and  $\mathbf{M}$ .
2:  $h^2 \leftarrow$  time step squared.
3: for each spring  $k$  connecting nodes  $(i, j)$  do
4:   for each coordinate  $d \in \{x, y, z\}$  do
5:      $l_{3i+d, 3i+d} += k_k$ ,  $l_{3j+d, 3j+d} += k_k$ 
6:      $l_{3i+d, 3j+d} -= k_k$ ,  $l_{3j+d, 3i+d} -= k_k$ 
7:   end for
8: end for
9: for each spring  $k$  connecting nodes  $(i, j)$  do
10:  for each coordinate  $d \in \{x, y, z\}$  do
11:     $j_{3i+d, 3k+d} += k_k$ ,  $j_{3j+d, 3k+d} -= k_k$ 
12:  end for
13: end for
14: for each node  $i$  do
15:  for each coordinate  $d \in \{x, y, z\}$  do
16:     $m_{3i+d, 3i+d} \leftarrow m_i$ 
17:  end for
18: end for
19: Precompute  $\mathbf{A} = \mathbf{M} + h^2\mathbf{L}$  and factorize for solving.
```

3.3.2 Local Update

The local update relies on local information for each spring. Specifically, it computes the spring direction vectors for each spring in the system, ensuring that they maintain their rest length constraint. Algorithm 2 details the local update step.

Algorithm 2 Local Update for Mass-spring System

Require: Current positions $\mathbf{p}$, rest lengths r_{ij} of springs

Ensure: Updated spring direction vectors $\mathbf{d}_{ij}$

```
1:  $j \leftarrow 0$ 
2: for each spring  $(i, j) \in$  system's spring list do
3:   Compute displacement vector:  $\mathbf{p}_{12} \leftarrow \mathbf{p}_i - \mathbf{p}_j$ 
4:   Normalize the displacement:  $\hat{\mathbf{p}}_{12} \leftarrow \frac{\mathbf{p}_{12}}{\|\mathbf{p}_{12}\|}$ 
5:   Compute updated spring direction:  $\mathbf{d}_{ij} \leftarrow r_{ij} \cdot \hat{\mathbf{p}}_{12}$ 
6:   Store  $\mathbf{d}_{ij}$  in spring direction list
7:    $j \leftarrow j + 1$ 
8: end for
```

3.3.3 Global Update

The global update, on the other hand, involves all springs simultaneously. In this step, we solve for $\mathbf{x}$ while keeping $\mathbf{d}$ fixed. We start from the quadratic minimization problem described in equation (2.15). To find the optimal $\mathbf{x}$, we take the gradient with respect to $\mathbf{x}$ and set it to zero:

$$\frac{\partial}{\partial \mathbf{x}} \left(\frac{1}{2} \mathbf{x}^T (\mathbf{M} + h^2 \mathbf{L}) \mathbf{x} - h^2 \mathbf{x}^T \mathbf{J} \mathbf{d} + \mathbf{x}^T \mathbf{b} \right) = 0 \quad (3.1)$$

Since $\mathbf{L}$ is symmetric and positive semi-definite, the system matrix $\mathbf{M} + h^2 \mathbf{L}$ is symmetric positive definite, this simplifies to:

$$(\mathbf{M} + h^2 \mathbf{L}) \mathbf{x} - h^2 \mathbf{J} \mathbf{d} + \mathbf{b} = 0 \quad (3.2)$$

Rearranging, we obtain:

$$(\mathbf{M} + h^2 \mathbf{L}) \mathbf{x} = h^2 \mathbf{J} \mathbf{d} - \mathbf{b}. \quad (3.3)$$

This is a convex quadratic system where $\mathbf{M}$ represents the mass matrix, $h^2 \mathbf{L}$ accounts for stiffness from the implicit Euler integration, $h^2 \mathbf{J} \mathbf{d}$ encodes local deformation constraints, and $\mathbf{b}$ contains external forces and previous state dependencies.

Since the system matrix $\mathbf{M} + h^2 \mathbf{L}$ is symmetric positive definite, this guarantees the existence of a unique solution, which we solve efficiently using Cholesky factorization. By precomputing this factorization (since the system matrix remains constant when mass, timestep, and connectivity do not change), we significantly speed up the computation in each iteration. Algorithm 3 details the global update step.

Algorithm 3 Global Update Step

Require: Precomputed matrices $\mathbf{M}$, $\mathbf{L}$, $\mathbf{J}$ and external forces $\mathbf{f}_{\text{ext}}$.

1: Compute the right-hand side:

$$\mathbf{b} = h^2 \mathbf{f}_{\text{ext}} - \mathbf{M} \mathbf{y}.$$

2: Compute $\mathbf{x}$:

$$\mathbf{x} = (\mathbf{M} + h^2 \mathbf{L})^{-1} (h^2 \mathbf{J} \mathbf{d} - \mathbf{b}).$$

3: Update the system state with $\mathbf{x}$.

3.3.4 Convergence Check

For ease of implementation, our simulation assumes convergence after a fixed number of iterations (e.g., 100). However, more advanced stopping criteria could be employed to adaptively determine convergence based on error thresholds or energy minimization.

Algorithm 4 Mass-Spring Solver with Fixed Iterations

Require: Maximum iterations n

- 1: Store previous state: $\mathbf{p}_{n-1} \leftarrow \mathbf{p}_n$
 - 2: **for** $i = 1$ to n **do**
 - 3: **Local Step:** Update spring directions $\mathbf{d}_{ij}$ by normalizing displacement vectors
 - 4: **Global Step:** Solve for $\mathbf{x}$ using the precomputed Cholesky factorization
 - 5: **end for**
-

Chng 4

Results and Discussion

4.1 Experimental Setup

For performance evaluation, all experiments were conducted on an AMD Ryzen 7 7840H processor with Radeon 780M Graphics. To ensure consistency, auto-parallelization was disabled, and all computations were performed on a single thread. Following the sub-stepping strategy [32], we set the time step to $h = 1/60$ seconds, corresponding to a 60 FPS simulation rate. As for external forces, we only consider gravitational acceleration $g = 9.8m/s^2$ [23] and user-applied forces, which arise from direct interactions such as pulling, pushing, or manipulating the system.

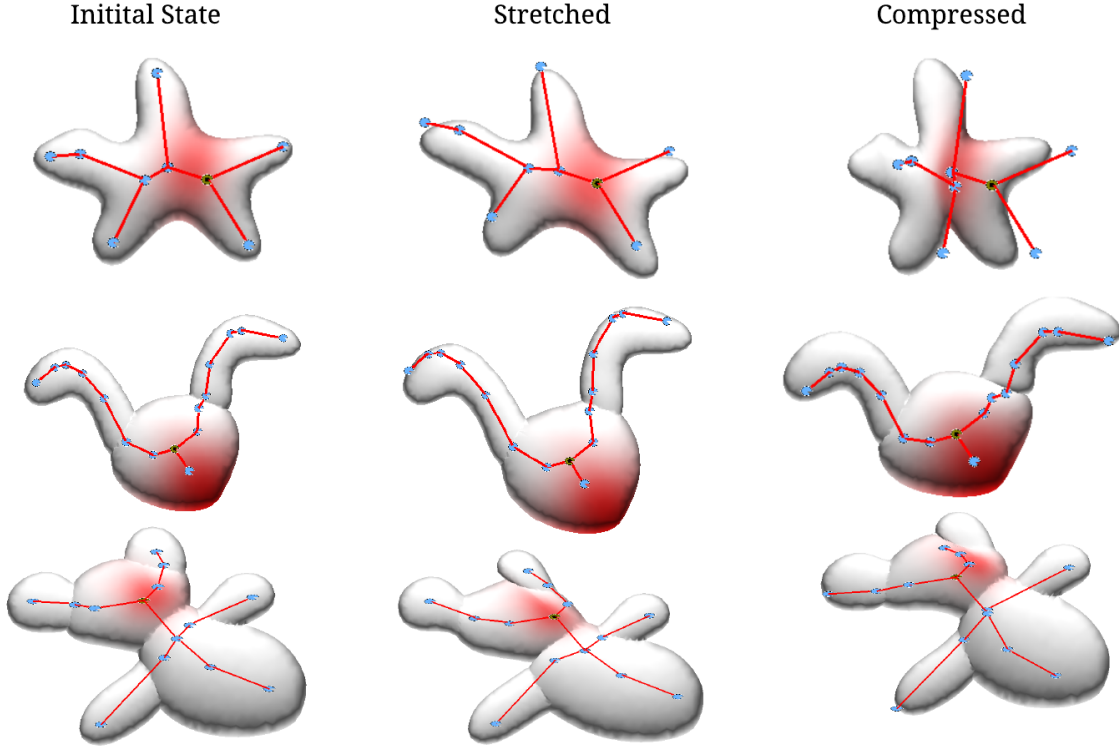
As for user interaction, the object responds to physics forces only when the user moves a joint with the mouse, allowing for precise control over movement. However, a limitation of this approach is that forces are not applied continuously, meaning the object lacks ongoing secondary motion unless actively manipulated. As a result, animation techniques like follow-through, slow-in, slow-out [33] may appear less pronounced.

4.2 Dynamic Motion Effects

Figure 4.1 illustrates the dynamic deformation response of our method under different external forces. The leftmost objects represent the initial state before any deformation is applied. The green dot marks the central joint, which, when moved, causes the entire object to follow its position, exhibiting dynamic effects.

When subjected to stretching forces (moving the mouse in one direction), the skeleton expands, causing the structure to elongate. Conversely, applying compression forces (moving the mouse in the opposite direction) causes the skeleton to contract. The stress distribution, visualized in red, highlights regions experiencing higher de-

formation. In contrast, without incorporating a mass-spring system into the bones, the object remains rigid and unchanged regardless of how the central joint is moved.



Hình 4.1: Dynamic deformation effects under different external forces. The leftmost column represents the initial state before deformation. The stretched column shows the effect of applying external forces that elongate the structure, while the compressed column illustrates the deformation when forces are applied in the opposite direction. The internal skeleton (red lines) adapts dynamically, and the stress distribution (highlighted in red) indicates regions experiencing high deformation.

However, a significant limitation of our method stems from the automatic BBW computation within the framework by [12]. The weight distribution is uneven, as evidenced by the red stress area being concentrated in a specific region rather than being evenly spread across the entire model. Moreover, users cannot manually adjust the weights or change the structure of the skeleton, which further restricts control over the deformation behavior. As a result, moving a blue joint does not propagate its influence effectively to other joints, reducing the clarity of localized deformations. While re-implementing BBW could enhance weight distribution and deformation accuracy, this is beyond the scope of our study. This limitation remains a key challenge in our approach.

Another limitation is that while we successfully introduce secondary animation,

the resulting motion is not as bouncy as observed in well-known frameworks such as Cartoon Animator 5 [19] and VRM [20]. This can be attributed to the absence of shear and bending springs in our implementation, as discussed in Section 3.1.

4.3 Effects of Stiffness Parameter

Stiffness determines how resistant a spring is to deformation, controlling how quickly it returns to its original shape after being stretched or compressed. The unit of stiffness is Newtons per meter (N/m) [23], meaning a higher stiffness value results in less deformation under the same force, leading to more rigid motion [13]. Therefore, adjusting the stiffness parameter is key to getting the best dynamic effects for each character. Since no pre-computation is needed, users can tweak stiffness in real time to find the most natural movement. As shown in Table 4.1, choosing the right stiffness is important. Because our system adds spring effects to bones for simple object animation (not for cloth or hair), lower stiffness values generally work better.

Stiffness Level (N/m)	Effect
Too Low (< 5)	Unstable, vibrating, or numerical errors in calculation.
Low ($5 - 10$)	Fast but may overshoot, slight jittering
Medium ($10 - 100$)	Balanced, elastic, natural-looking motion..
High (> 100)	Slow, bouncy motion is barely noticeable.

Bng 4.1: Effects of different stiffness levels on dynamic motion.

4.4 Complexity

The local update function, as described in 3.3.2, iterates over all springs in the system and updates their directions. The complexity for this step is therefore $\mathcal{O}(n)$.

The global update function, as described in ??, involves solving a linear system. Since we use Cholesky decomposition, the factorization of the $n \times n$ system matrix requires $\mathcal{O}(n^3)$ operations. Once factorized, solving for x using forward and backward substitution takes $\mathcal{O}(n^2)$ time. The overall complexity of the global update step is therefore $\mathcal{O}(n^3)$. Although this complexity may seem large, our system remains computationally feasible as it is applied to skeletal structures rather than dense meshes. In typical cases, the number of joints does not exceed $n = 1000$, leading to $n^3 = 1000^3 = 1,000,000,000$. Empirical tests show that for typical character skeletons ($n \leq 30$), the update process completes within milliseconds, ensuring smooth interactive performance. Thus, the computational cost is acceptable within our application domain.

Chng 5

Conclusion

In this work, we introduced a sketch-based 3D modeling system that integrates real-time skeletonization with mass-spring dynamics, enabling users to easily create soft-body effects for 3D characters. By leveraging the RealSkel [12] framework for automatic skeleton extraction and mass-spring physics [21] for skeletal dynamics, our system allows for the generation of dynamic secondary motion without the need for complex rigging or extensive animation expertise.

While the system successfully bridges sketch-based modeling and dynamic animation, there are several avenues for future work. One important direction is to refine the weight distribution process to achieve smoother and more consistent deformations across the model. Another key area is enabling continuous secondary motion without requiring user input, allowing the model to exhibit persistent dynamic effects. Additionally, expanding the system to include bend and shear springs would further enhance the flexibility and realism of the characters movements. To ensure that the system remains efficient and scalable, future work will also focus on improving computational performance, particularly for more complex models or detailed animations.

By addressing these challenges, we aim to enhance the systems capabilities, making it a more powerful and user-friendly tool, not only for animators but also for educational purposes in simulating materials.

References

- [1] T. Igarashi, S. Matsuoka, and H. Tanaka, *Teddy: A Sketching Interface for 3D Freeform Design*. New York, NY, USA: Association for Computing Machinery, 1 ed., 2023.
- [2] C.-L. Tai, H. Zhang, and J. Fong, “Prototype modeling from sketched silhouettes based on convolution surfaces,” *Comput. Graph. Forum*, vol. 23, pp. 71–84, 03 2004.
- [3] O. A. Karpenko and J. F. Hughes, “Smoothsketch: 3d free-form shapes from complex sketches,” *ACM Transactions on Graphics*, vol. 25, pp. 589–598, July 2006.
- [4] A. Nealen, T. Igarashi, O. Sorkine, and M. Alexa, “Fibermesh: designing freeform surfaces with 3d curves,” *ACM Trans. Graph.*, vol. 26, p. 41es, July 2007.
- [5] M. Sugihara, E. De Groot, B. Wyvill, and R. Schmidt, “A sketch-based method to control deformation in a skeletal implicit surface modeler,” in *Proceedings of the Fifth Eurographics Conference on Sketch-Based Interfaces and Modeling*, SBM’08, (Goslar, DEU), p. 6572, Eurographics Association, 2008.
- [6] M. Dvorožnak, D. Sykora, B. Curless, C. J. Curtis, O. Sorkine-Hornung, and D. H. Salesin, “Monster mash: A single-view approach to casual 3d modeling and animation,” (New York, NY, USA), pp. 1–12, 2020.
- [7] P. Borosán, M. Jin, D. DeCarlo, Y. Gingold, and A. Nealen, “Rigmesh: automatic rigging for part-based shape modeling and deformation,” vol. 31, Nov. 2012.
- [8] I. Baran and J. Popović, “Pinocchio: Automatic skeleton generation for 3d models,” *ACM Transactions on Graphics (TOG)*, vol. 26, no. 3, p. 33, 2007.
- [9] P. Borosán, T. Auzinger, and M. Wimmer, “Rigmesh: Automatic rigging for articulated characters,” *Computer Graphics Forum*, vol. 31, no. 2pt3, pp. 381–390, 2012.

- [10] Z. Xu, Y. Zhou, E. Kalogerakis, C. Landreth, and K. Singh, “Rignet: Neural rigging for articulated characters,” in *ACM Transactions on Graphics (TOG)*, vol. 39, p. 67, ACM, 2020.
- [11] J. Yang, X. Zuo, S. Wang, Z. Yu, X. Li, B. Ni, M. Gong, and L. Cheng, “Object wake-up: 3d object rigging from a single image,” *arXiv preprint arXiv:2108.02708*, 2021.
- [12] X. Ma, S. Xie, and K. Takayama, “Realskel: Real-time skeletonization for sketch-based modeling,” *Computer Graphics Forum*, vol. 40, no. 7, pp. 123–133, 2021.
- [13] A. W. Bargteil and T. Shinar, “An introduction to physics-based animation,” in *ACM SIGGRAPH 2019 Courses*, SIGGRAPH ’19, (New York, NY, USA), Association for Computing Machinery, 2019.
- [14] X. Provot, “Deformation constraints in a mass-spring model to describe rigid cloth behaviour,” in *Proceedings of Graphics Interface ’95*, GI ’95, (Toronto, Ontario, Canada), pp. 147–154, Canadian Human-Computer Communications Society, 1995.
- [15] R. Bridson, R. Fedkiw, and J. Anderson, “Robust treatment of collisions, contact and friction for cloth animation,” *ACM Trans. Graph.*, vol. 21, p. 594603, July 2002.
- [16] M. Teschner, B. Heidelberger, M. Müller, and M. Gross, “A versatile and robust model for geometrically complex deformable solids,” vol. 0, pp. 312–319, 07 2004.
- [17] A. Selle, M. Lentine, and R. Fedkiw, “A mass spring model for hair simulation,” *ACM Trans. Graph.*, vol. 27, p. 111, aug 2008.
- [18] R. E. Rosenblum, W. E. Carlson, and E. Tripp III, “Simulating the structure and dynamics of human hair: modelling, rendering and animation,” *The Journal of Visualization and Computer Animation*, vol. 2, no. 4, pp. 141–148, 1991.
- [19] Reallusion, “Cartoon animator 5 - spring bone system,” 2023. Available at <https://www.reallusion.com/cartoon-animator/spring-animation.html>.
- [20] VRM Consortium, “VRM Spring Bone Documentation,” 2024.
- [21] T. Liu, A. W. Bargteil, J. F. O’Brien, and L. Kavan, “Fast simulation of mass-spring systems,” *ACM Trans. Graph.*, vol. 32, Nov. 2013.
- [22] O. Aichholzer and F. Aurenhammer, “Straight skeletons for general polygonal figures in the plane,” *Lecture Notes in Computer Science*, vol. 1090, 06 1997.
- [23] D. Halliday, R. Resnick, and J. Walker, *Fundamentals of Physics*. Halliday & Resnick Fundamentals of Physics, John Wiley & Sons Canada, Limited, 2010.

- [24] S. Martin, B. Thomaszewski, E. Grinspun, and M. Gross, “Example-based elastic materials,” *ACM Trans. Graph.*, vol. 30, July 2011.
- [25] D. Baraff and A. Witkin, *Large Steps in Cloth Simulation*. New York, NY, USA: Association for Computing Machinery, 1 ed., 2023.
- [26] L. Paul Chew, “Constrained delaunay triangulations,” *Algorithmica*, vol. 4, p. 97108, June 1989.
- [27] A. Jacobson, I. Baran, J. Popović, and O. Sorkine, “Bounded biharmonic weights for real-time deformation,” *ACM Trans. Graph.*, vol. 30, July 2011.
- [28] S.-W. Hsiao and R.-Q. Chen, “A method of drawing cloth patterns with fabric behavior,” in *Proceedings of the 5th WSEAS International Conference on Applied Computer Science*, ACOS’06, (Stevens Point, Wisconsin, USA), p. 635640, World Scientific and Engineering Academy and Society (WSEAS), 2006.
- [29] The Qt Company, *Qt Documentation*, 2023.
- [30] Kitware, Inc., “Cmake: Cross-platform make,” 2025.
- [31] G. Guennebaud, B. Jacob, *et al.*, “Eigen: A c++ template library for linear algebra,” 2023.
- [32] M. Macklin, K. Storey, M. Lu, P. Terdiman, N. Chentanez, S. Jeschke, and M. Müller, “Small steps in physics simulation,” in *Proceedings of the 18th Annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, SCA ’19, (New York, NY, USA), Association for Computing Machinery, 2019.
- [33] F. Thomas and O. Johnston, *Disney Animation: The Illusion of Life*. New York: Abbeville Press, 1981.